SIMATS ENGINEERING

OPERATING SYSTEMS

ASSIGNMENT – CO4

OPTIMISING FILE STORAGE AND DISK ACCESS

Course: Operating Systems (CSA04)

Course Outcome: CO4 – Optimise various file allocation strategies and disk scheduling

algorithms to identify optimal methods for enhancing system performance (file systems, file

allocation methods, modern file systems, distributed and cloud-based storage, disk scheduling

and device management)

Nature of Assignment: Individual

Duration: One Week

Total Marks: 50

Submission Date:

Bloom's Levels Assessed: K5 – Evaluate (every task in this assignment is at K5 level)

Mandatory Execution and GitHub Submission

• Every student must individually execute every task in this assignment, including all

"Complete the Code" programs.

• Create a GitHub repository named CSA04-<RegisterNumber>-Assignment4 and share

it with the course faculty (public, or private with the faculty added as a collaborator).

• Keep one folder for each task (for example: Task01, Task02 … or Q1, Q2 …), containing

the C source files, shell scripts, configuration files (Dockerfile, YAML), command outputs

and screenshots.

• Include a README.md with your name, register number, compiler and version (for

example gcc 13), the Linux distribution and version used, the exact gcc command used

to compile (for example gcc -std=c11 -Wall -Wextra -pthread), the steps to run each

program, and a short note of the result of each task.

• Commit your work progressively – at least one commit for each task. The commit history

will be checked; a single upload of all files on the last day will not be accepted.

• Submit the GitHub link below and on the cover page of your report. An assignment

submitted without a working GitHub link will not be evaluated.

GitHub Repository Link: https://github.com/192524372simats/CSA0413-OS.git

How to Answer the Additional Questions

Complete the Code: Copy the given code into your repository, replace every TODO with

working code without changing the rest of the program, execute it and compare your output (or

result) with the expected output.

Alternative Concept and Complexity Analysis: Describe how the solution would change if the

alternative concept were used, with code or pseudocode, and analyse and compare the time

complexity and space complexity of both solutions using Big-O notation, supported by your own

measurements wherever possible.

Case study and overall design

The university result portal has low normal-day demand but a sharp burst of 150,000 student logins in two hours. A single physical server is a single point of failure and cannot reliably absorb the burst. The proposed design uses a hybrid approach: retain suitable university infrastructure for baseline services and use public-cloud capacity for the result-day burst. A load balancer distributes requests across stateless portal replicas. A managed database, backups, monitoring and tested recovery procedures protect the data and availability.

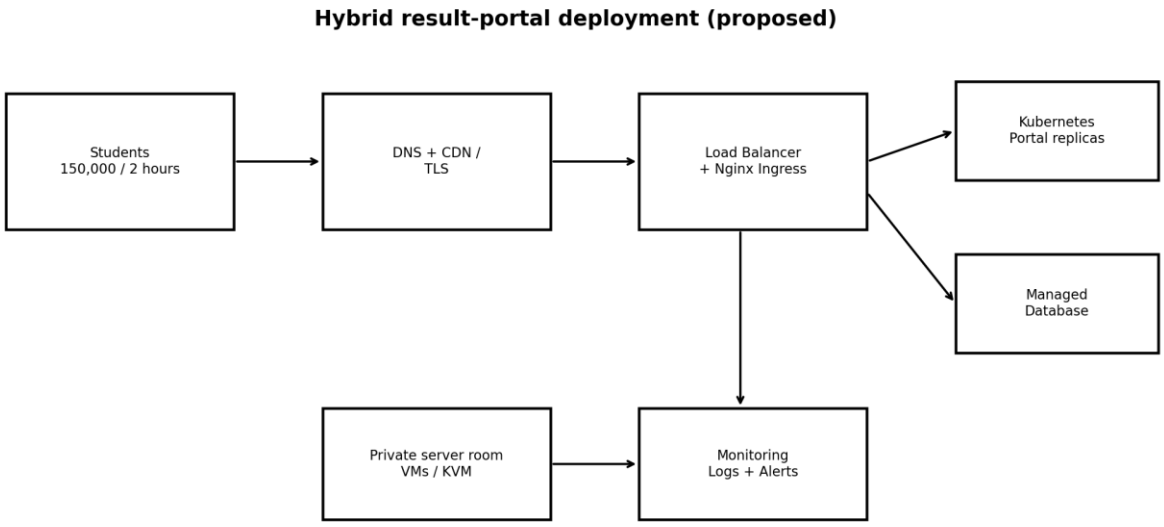


Figure 1. Proposed hybrid architecture (conceptual design).

PART A – DESIGN

TASK A1 – Evaluate Linux system components

A1.1 Linux architecture

Linux is commonly described as a kernel surrounded by user-space components. The kernel is the privileged core that manages CPU scheduling, virtual memory, filesystems, device drivers, networking and system calls.

Applications request kernel services through system calls. System libraries (for example, glibc) provide reusable APIs that wrap many system calls. The shell (such as Bash) interprets commands, supports scripting and connects utilities through pipes and redirection. Utilities (for example, ls, ps, grep, systemctl and ip) perform administration and user tasks.

For the portal, the scheduler allocates CPU time to web workers; virtual memory and page cache reduce disk reads; the VFS gives applications a common interface to filesystems; device drivers handle disks and network interfaces; and the network stack processes TCP/IP traffic. Linux is suitable because it is automatable, observable, widely supported by cloud platforms and can run the same application stack in VMs and containers.

A1.2 Process and memory management

The Completely Fair Scheduler (CFS; modern Linux kernels are transitioning scheduling internals toward EEVDF) distributes CPU time among runnable tasks using fairness accounting and scheduling weights. A web worker with higher scheduling weight can receive a larger share, but CPU quotas and sensible worker counts are still required to prevent one service starving others. CFS improves fairness; it does not guarantee low latency under overload.

The page cache stores recently accessed file data in RAM, reducing repeated disk I/O and improving static-file and database performance. Linux reclaims cache when applications need memory. Swapping moves less-active memory pages to disk; it can prevent immediate out-of-memory failure but excessive swapping causes severe latency. For a latency-sensitive result portal, monitor swap-in/swap-out and avoid relying on swap as extra RAM.

/proc exposes process and kernel runtime information (for example /proc/meminfo, /proc/cpuinfo and /proc/<pid>/status). /sys exposes device,

driver and kernel-object attributes. They are virtual filesystems, not ordinary disk directories. ps/top, free, vmstat, iostat and ss complement these interfaces. Evaluation: these facilities provide low-cost, scriptable observability, but administrators must interpret trends rather than treating one snapshot as a capacity guarantee.

A1.3 Linux filesystem and permissions

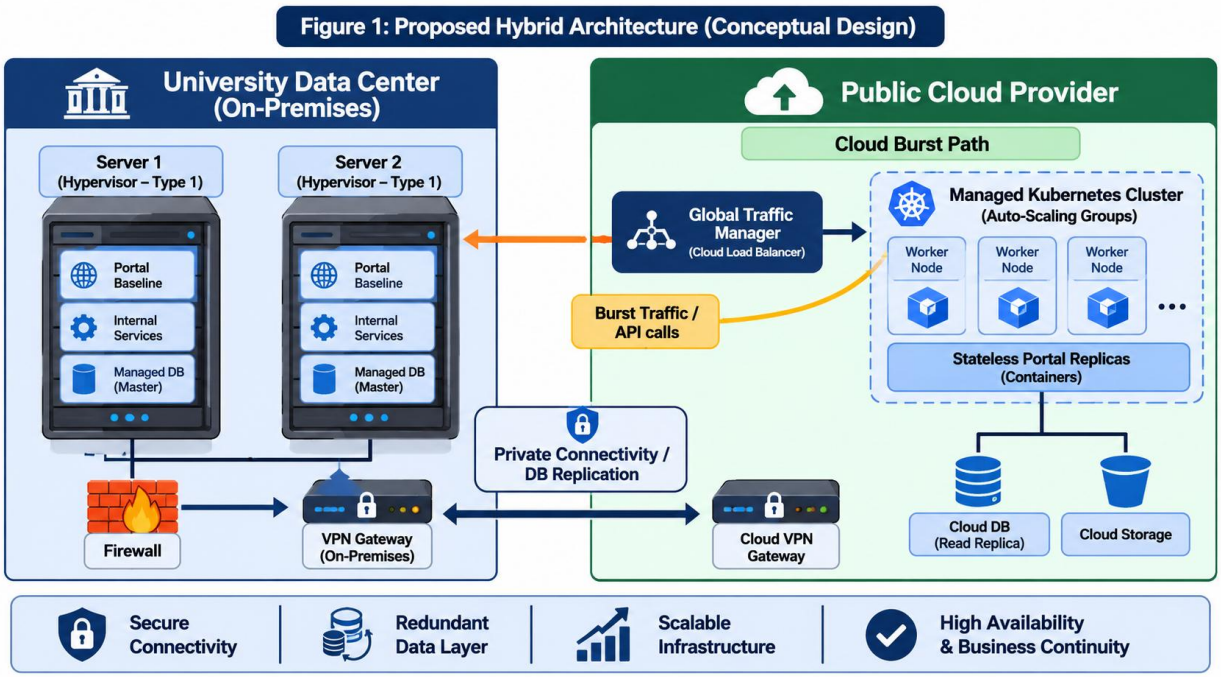
The Linux directory tree starts at /. Common directories include /etc (configuration), /home (user files), /var/log (logs), /var/lib (service state), /usr (programs and libraries), /tmp (temporary files), /dev (device nodes), /proc and /sys (kernel interfaces), and /boot (boot files). The VFS provides a common abstraction for filesystems such as ext4 and XFS, allowing applications to use common open/read/write operations.

Each file has an owner, group and permission bits for owner/group/others: read, write and execute. Directories use execute permission to allow traversal. Use least privilege: a dedicated portal account should own application files and should not be an administrator. setuid runs a program with the file owner's effective identity and must be used sparingly because unsafe setuid programs can create privilege-escalation risks. sudo grants approved administrative commands with auditing; it is preferable to routine root login.

Alternative: Windows Server

On Windows Server, the portal could run under IIS with a supported Python hosting arrangement or in containers, while administration uses PowerShell, Windows services, Event Viewer and Windows Firewall. A typical Windows base installation generally consumes more disk and memory than a minimal Linux server installation, although exact values depend on edition, features and updates and must be measured on the selected versions. PowerShell and Group Policy can automate administration, but Linux shell

tooling may be more direct for a Python/Linux stack. Windows Server licensing is edition-, core- and deployment-dependent and may include CAL considerations; Linux distributions can be used without per-server OS licence fees, although support subscriptions and application licences may apply. For this case, Linux is preferred for lower baseline footprint, mature cloud/container tooling and automation flexibility. A fair comparison should record actual idle RAM and disk after installation and obtain a current vendor quote rather than invent a licence price.



TASK A2 – Design system administration

A2.1 Users, services and logs

Create a group named portal and a non-login service account named portal. Keep administrators in a separate sudo-enabled group. The portal process should read application code and write only to designated upload/cache directories. Run the application through systemd with Restart=on-failure, a controlled working directory, environment file permissions and a dedicated user. The example portal.service is included in Task01. Send application

logs to stdout/stderr so systemd-journald can collect them; use journalctl -u portal.service and persistent journal storage. Configure log rotation and alerts for repeated 5xx errors, authentication failures, disk usage and service restarts.

A2.2 Networking services

Use a private subnet with a reserved private IP for each server; avoid hard-coding public addresses. Create a DNS name such as results.university.example pointing to the public load balancer. Nginx acts as a reverse proxy, terminates TLS where appropriate, forwards requests to the application and adds forwarding headers. Permit inbound TCP 80/443 to the proxy and SSH TCP 22 only from approved administrator IP ranges or a VPN. Deny other unsolicited inbound traffic. Use SSH public-key authentication, disable password/root SSH login after verifying key access, and keep packages patched. Example Nginx configuration is included in Task01.

A2.3 Automation

Automate nightly database backups with retention and restore tests, hourly log analysis, log rotation, disk-space checks, certificate-expiry checks, package/security updates in a maintenance window and service-health checks. Cron is suitable for simple periodic jobs; systemd timers provide better integration with service dependencies and logs. The included disk_alert.sh checks root filesystem usage. Schedule scripts using absolute paths and redirect output to a monitored log.

Alternative: Ansible vs shell scripts

For one server, a short shell script may be quicker to write and run. For 20 servers, Ansible playbooks are usually more efficient because the same declarative tasks can be applied consistently over SSH, with inventory variables and reusable roles. Hand-written scripts can drift when commands,

paths or execution order differ between machines. Ansible adds an initial learning and role/inventory maintenance cost, but reduces configuration drift and makes changes reviewable. Complexity of a single local script is roughly O(k) in the number of steps; applying it manually across n servers is O(nk) operator executions. Ansible still performs work on n hosts (roughly O(nk) total operations), but parallel execution reduces elapsed wall-clock time and centralises control. Actual timings should be measured for 1 and 20 test VMs.

Figure 2: Linux System Components and Observability

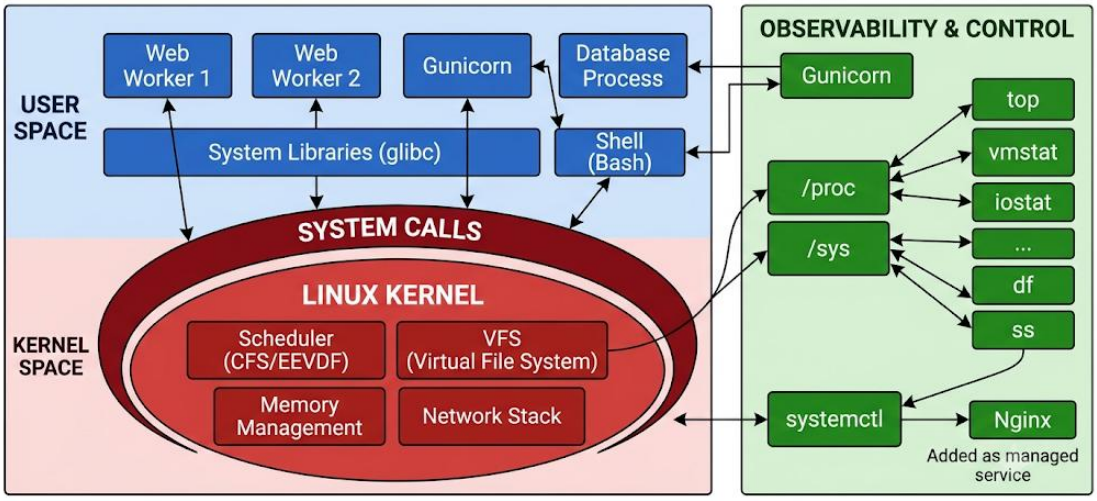


Figure 2: Linux System Components and Observability

TASK A3 – Evaluate virtualisation

A3.1 Hypervisors

A type-1 hypervisor runs directly on server hardware and manages guest VMs; examples include KVM (integrated with the Linux kernel), VMware ESXi, Xen and Microsoft Hyper-V. A type-2 hypervisor runs as an application on a host operating system; examples include VirtualBox and VMware Workstation. Full virtualisation presents virtual hardware so an unmodified guest OS can run. Para-virtualisation uses guest awareness or specialised interfaces/drivers to reduce virtualisation overhead. Intel VT-x

and AMD-V provide hardware CPU virtualisation support; IOMMU features assist device assignment.

A3.2 Benefits and costs

VMs isolate guest operating systems, consolidate several services on one physical host, support snapshots for testing/rollback and may support live migration between compatible hosts. They also consume memory for each guest OS, add CPU/I/O overhead, require storage and management, and may incur hypervisor or guest OS licence costs. For the university, VMs are useful for separating legacy systems and administrative domains; stateless portal replicas are better suited to containers.

Alternative: type-2 VirtualBox vs type-1 KVM

A type-2 hypervisor adds the host OS and hosted virtualisation layer. CPU-bound workloads may incur extra scheduling/context-switch overhead; I/O-bound workloads may pass through additional host storage/network layers. The host OS consumes RAM that is unavailable to guests. VirtualBox is convenient for student labs and desktop testing, but a type-1 platform such as KVM is generally more appropriate for production server consolidation, automation and performance control. Benchmark the same VM workload on the same hardware before quantifying overhead.

Figure 3: Virtualization vs. Containerization (Architecture Comparison)

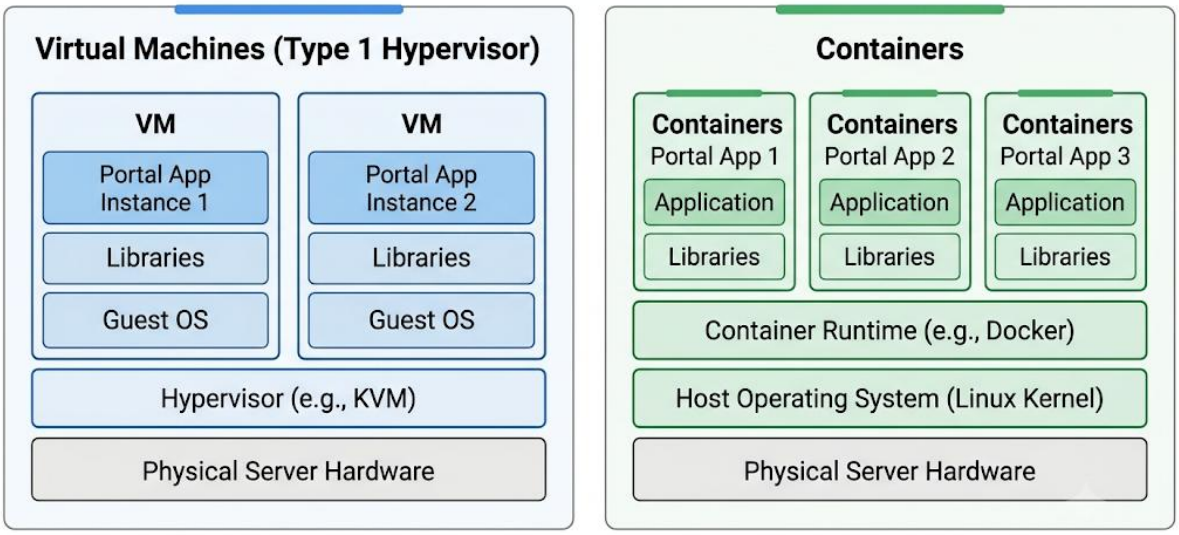


Figure 3: Virtualization vs. Containerization (Architecture Comparison)

TASK A4 – Evaluate containerisation

A4.1 How containers work

Linux namespaces isolate views of resources such as process IDs, network interfaces, mount points, hostnames and user IDs. Control groups (cgroups) account for and limit CPU, memory and I/O resources. A container is a group of processes using the host kernel, not a complete guest OS. An image is assembled from read-only layers; each Dockerfile instruction can add a layer, while a writable container layer sits above the image. Layer caching speeds rebuilds when earlier inputs do not change.

A4.2 Containers versus VMs

Containers generally start in seconds or less and use less memory because they share the host kernel. VMs commonly take longer to boot and need RAM for a full guest OS, but provide a separate kernel boundary and can run different operating systems. Containers provide process/resource isolation, but share the host kernel, so kernel vulnerabilities and unsafe privileges can affect multiple containers. Use containers for stateless portal replicas and supporting services; use VMs where stronger OS-level

isolation, different kernels or legacy OS requirements are necessary. Apply non-root users, read-only filesystems where possible, minimal images, capability dropping, patching and resource limits.

Alternative: separate VMs for portal and database

Running the portal and database in separate VMs gives each a guest OS and stronger kernel separation, but each instance needs guest memory and storage and takes longer to boot (typically minutes rather than container seconds, depending on configuration). Containers share the host kernel and therefore have a smaller per-instance footprint. VMs may be preferable for a database with strict isolation requirements; containers are more efficient for rapidly scaling stateless web replicas.

TASK A5 – Cloud deployment and orchestration

A5.1 Cloud set-up

IaaS supplies virtual machines, networks and storage while the customer manages the OS and application. PaaS manages more of the runtime/platform so developers focus on application code. SaaS delivers a complete application to end users. For result day, use a public-cloud load balancer, autoscaled Kubernetes worker capacity or managed Kubernetes, stateless portal pods, a managed relational database with backups and high availability, object storage for static documents, CDN caching, private networking, secrets management and monitoring. Keep sensitive database access private and encrypt traffic and backups.

A5.2 Kubernetes

A Pod is the smallest deployable unit and contains one or more tightly coupled containers. A Deployment maintains the desired number of replicas and replaces failed pods. A Service gives stable network access to matching pods. Resource requests guide scheduling; limits cap resource consumption.

A Horizontal Pod Autoscaler (HPA) changes replica count based on metrics such as CPU utilisation or custom request/latency metrics. Readiness probes prevent traffic reaching unready pods. Replicas spread across nodes/zones improve availability, while database replication and backups protect persistent data.

Alternative: fixed VMs vs Kubernetes autoscaling

Fixed VMs sized for peak demand can respond immediately if already running, but leave substantial capacity idle during most of the year and incur ongoing cost. Kubernetes autoscaling can add replicas and nodes as demand increases, improving utilisation and cost efficiency. Scaling is not instantaneous: image pulls, node provisioning, readiness checks and database bottlenecks introduce delay. Pre-scale before the published result window, maintain warm capacity and load-test. Autoscaling reduces waste but requires operational expertise and correctly configured metrics/limits.

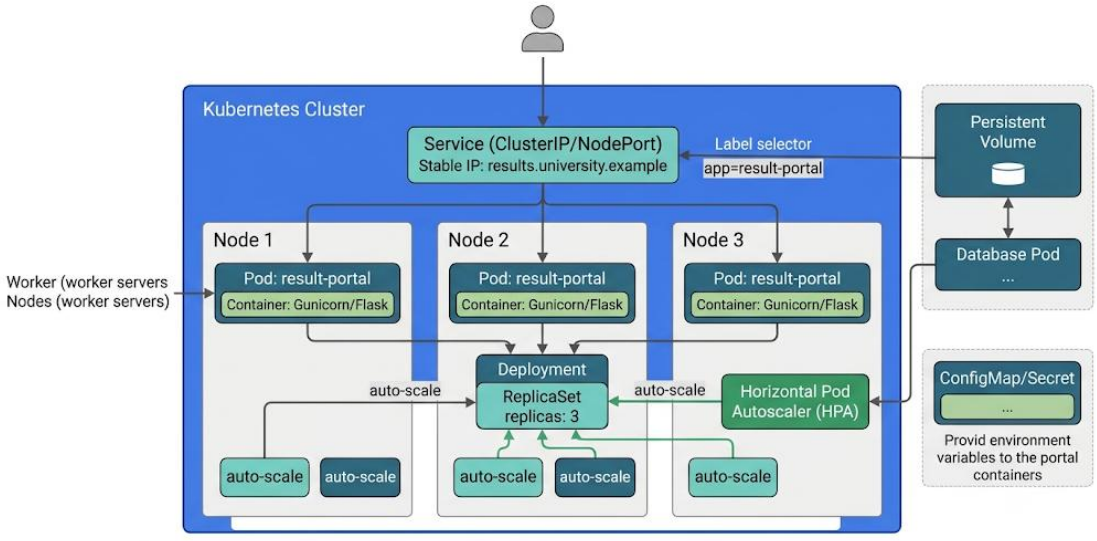


Figure 4: Kubernetes Pod, Deployment, and Service (Orchestration Logic)

PART B – IMPLEMENTATION & EXPERIMENTAL INVESTIGATION

The files supplied with this report are runnable examples. Execute them on your own Linux/VM environment and paste the actual command outputs and screenshots into the task folders. The assignment explicitly requires execution evidence; this report does not fabricate machine-specific measurements.

TASK B1 – Administer the Linux server

B1.1 Observe the system

Run: ps aux --sort=-%cpu | head; top; free -h; vmstat 1 5; iostat -xz 1 3; df -h; ss -tulpn. ps/top show process CPU and memory; free reports physical memory and cache/buffers; vmstat reports runnable/blocked processes, memory, swap, I/O and CPU; iostat reports device throughput, queueing and utilisation; df reports filesystem capacity; ss shows listening sockets and connections. Install sysstat for iostat if unavailable. Record timestamp, host specs and outputs in your report.

B1.2 Automate

Task01/log_summary.sh uses awk to extract status codes and client IPs, sort and uniq -c to count them, then sorts by code or descending frequency. The expected result from the supplied seven-line sample is status counts 200=4, 302=1, 403=1 and 404=1; busiest client 10.0.0.5 has 4 requests; total bytes are 43,200. The included disk_alert.sh warns at 80% root filesystem use. Example hourly cron entry: 0 * * * * /bin/bash /absolute/path/Task01/log_summary.sh >> /absolute/path/log-summary.out 2>&1.

B1.3 Services and network

Create a dedicated portal user, install the app under /opt/result-portal, create a Python virtual environment, install Flask and Gunicorn, copy Task01/portal.service to /etc/systemd/system/portal.service, then run systemctl daemon-reload, systemctl enable --now portal.service and systemctl status portal.service. Configure Nginx with Task01/nginx.conf, enable the site and reload Nginx. With UFW, allow OpenSSH, 'Nginx Full', then enable the firewall. Restrict SSH to a trusted IP/VPN in production. Capture systemctl, nginx -t, ufw status and curl outputs.

B1.4 Compare with design

The implementation uses a sample Flask service and a local reverse proxy rather than a production multi-node deployment. It validates service management, reverse proxying and automation concepts. Production still requires TLS certificates, DNS, database integration, centralised monitoring, backups and high availability.

Complete the Code – B1

Completed script: Task01/log_summary.sh. It preserves the given sample data and implements both TODOs. TODO 1: awk '{print $7}' | sort | uniq -c | awk '{print $2, $1}' | sort -n. TODO 2: awk '{print $1}' | sort | uniq -c | sort -rn | head -1 | awk '{print $2, $1}'. Run with bash Task01/log_summary.sh.

Alternative: central logging

A central system such as the ELK stack or a central journald/log server collects logs from all 10 servers into one searchable location. This can reduce cross-server error investigation from manually logging into each host to one indexed query, at the cost of network traffic, storage, ingestion CPU/RAM and setup/maintenance effort. Shell scripts are lightweight and simple for a few hosts; central logging becomes more valuable as server

count and incident volume grow. Retention, access control and sensitive-data redaction are essential.

TASK B2 – Containerise the portal

Task02 contains app.py, requirements.txt and a completed Dockerfile. TODO 1 is WORKDIR /app. TODO 2 is COPY --chown=portal:portal app.py . and USER portal (with the portal account created in the preceding RUN). TODO 3 is EXPOSE 8000 and a Gunicorn command binding 0.0.0.0:8000 with app:app. Build using docker build -t result-portal . and run using docker run -d --name result-portal -p 8000:8000 --cpus=0.5 --memory=512m result-portal. Check docker ps, curl localhost:8000, docker images, docker exec result-portal whoami, docker stats --no-stream and docker history result-portal.

The Dockerfile installs dependencies before copying source so dependency layers can be cached. python:3.12-slim reduces the base image footprint compared with a full Ubuntu image. Running as portal reduces privileges. EXPOSE documents the service port; it does not publish it by itself, so -p is still needed. The Compose file runs a portal and PostgreSQL with a named volume and CPU/memory limits. The demo Flask route is not yet database-connected; it demonstrates persistent DB volume configuration rather than a complete result database.

Alternative: Ubuntu base or multi-stage build

A full Ubuntu base generally includes more packages and produces a larger image, increasing transfer time and potential attack surface. A slim Python base is smaller for this Python service. Multi-stage builds are especially useful when compilation tools are needed: build dependencies in a builder stage and copy only runtime artifacts into the final stage. Compare actual docker images sizes, build duration and pull duration on the same network.

Fewer packages generally mean fewer potential vulnerabilities, but scanning and patching remain necessary.

TASK B3 – Compare bare metal, VM and container

B3.1 Deploy three ways

Use the same Flask application and endpoint for all three tests. Bare metal: create a venv on Linux and run Gunicorn directly. VM: install the same Linux distribution and app in VirtualBox/KVM, then run Gunicorn. Container: build and run Task02/Dockerfile. Keep app version, worker count, CPU allocation, memory allocation and test client consistent. Record the exact configuration.

B3.2 Measure

Measure startup time from command invocation until /health returns HTTP 200; memory after warm-up using free/ps/docker stats or VM host metrics; disk/image size using du -sh and docker images; and throughput with ApacheBench, wrk or hey. Example: ab -n 10000 -c 100 http://HOST:8000/. Repeat at least three times and report median plus test conditions. Use a table with columns Deployment, Startup (s), Memory (MiB), Disk/image (MiB), Requests/s, p95 latency and notes. Fill only from your measurements.

B3.3 Scale with Kubernetes

Build the image and load it into minikube or kind, then apply Task03/deployment.yaml. It requests 3 replicas and defines CPU/memory requests and limits. Verify kubectl get pods; delete one pod and observe the Deployment recreate it; scale to 6 with kubectl scale deployment/result-portal --replicas=6. Apply Task03/second-service.yaml to add an independently limited Nginx-based LMS demo service. Note that the

provided Service is NodePort for local clusters; a cloud LoadBalancer service can be used in a cloud cluster.

B3.4 Engineering trade-off

Bare metal avoids a guest OS and container runtime layer and can offer strong raw performance, but has weaker workload portability and requires physical failover planning. VMs provide OS-level isolation and flexible consolidation but consume more memory/storage. Containers are lightweight, reproducible and fast to scale, but share the host kernel. Kubernetes adds scheduling, health recovery and multi-node orchestration but increases operational complexity. Use bare metal for specialised high-performance workloads, VMs for isolation/legacy systems and containers/Kubernetes for scalable stateless portal services.

Complete Kubernetes code: Task03/deployment.yaml sets replicas: 3 and selector.matchLabels.app: result-portal. TODO 2 sets requests cpu: 250m, memory: 256Mi and limits cpu: 500m, memory: 512Mi.

Alternative: Docker Compose scaling vs Kubernetes

Compose can run six replicas on one host, which is simple for development or a small single-server deployment. It cannot provide node-level failover if that host fails, and the host's CPU/RAM/network capacity is the upper bound. Kubernetes can distribute replicas across nodes and recreate failed pods, improving resilience and scaling capacity, but requires a cluster and more operational effort. Compose may recover a failed container on the same host; it cannot recover from loss of the only host.

TASK B4 – Final engineering decision

B4.1 Practical extension

Task03/second-service.yaml adds a second independent service named lms-demo, with its own Deployment, Service, CPU request/limit and memory

request/limit. Apply it and capture kubectl get deployments,pods,svc and kubectl describe pod outputs. It is a demonstration service (Nginx), not a full learning-management system or examination platform.

B4.2 Capacity estimate

The demand average is 150,000 / (2 × 3,600) = 20.83 student logins per second. This is an average arrival rate, not the peak request rate: each login may generate several HTTP/database requests and arrivals may be bursty. If your measured sustainable throughput per container is R requests/second and the estimated peak application request rate is P requests/second, base replicas = ceil(P/R). Add headroom (for example 30–50%) and account for database throughput, cache hit ratio, session storage, file downloads and autoscaling delay. If only a login rate L is known and each login causes q requests, estimate P = L × q. Replace R and q with measured/load-tested values; do not treat the 20.83 average as a safe sizing target.

B4.3 Production decision

Recommended option: hybrid deployment using university-owned KVM VMs/containers for normal workloads and a public-cloud burst path for the result-day peak, with Kubernetes or a managed container platform for stateless replicas. Place the database on a managed highly available service or a carefully operated private database with tested replication/backups, depending on data residency and budget. This avoids purchasing peak-sized physical servers that sit idle most of the year while retaining control of core university systems. Before production, conduct a full-scale load test, validate cloud egress/database costs, establish a data-processing agreement, encrypt student data, restrict access, test restore/failover and pre-scale before result publication.

Alternative: all-public-cloud pay-per-hour

An all-cloud design can add capacity quickly through autoscaling and avoids buying/maintaining physical servers, but costs continue for baseline compute, database, storage, backups, load balancers, monitoring and network egress. With two result days per year, peak capacity can be provisioned temporarily, but baseline services still run year-round. Annual estimate should be calculated from provider rates: (baseline hourly resources × 8,760) + (incremental peak resources × peak hours per year) + database/storage/network/support. Cloud provider infrastructure controls physical hosting; the university remains responsible for data governance, identity, access, encryption and contractual choices. Exact cost and data-location control depend on provider, region, service and contract; obtain a current quote.

Final conclusion

Linux provides an efficient, observable and automatable operating environment. VMs are useful for isolation and consolidation, while containers are better suited to fast, repeatable deployment of stateless application replicas. Kubernetes adds health recovery and horizontal scaling, but it must be combined with load testing, database capacity planning, monitoring and security controls. A hybrid model is the most balanced design for the university because it uses existing infrastructure for normal demand and cloud elasticity for the exceptional result-day burst.